

Trinity College Dublin
Computer & Electronics Engineering

3C07 DIGITAL SYSTEMS DESIGN

ASSIGNMENT 2 REPORT

LFSR, FSM and Counter to Detect a Codeword

Student Name
Paula Brüdrieh

Student ID
23375364

Module Coordinator:

Dr. Shreejith Shanker

Submission Date : 12.04.26

Contents

1	Implementation Notes	2
1.1	Adapting the LFSR	2
1.2	Finite State Machine	2
1.3	Adapting the Counter	4
1.4	A Note on Clocks	4
2	Writing the Top Module	4
3	Test Plan	5
3.1	Scope	5
3.2	Methodology	5
3.3	Features of Test Bench	6
3.4	Test Cases	6
3.4.1	Initial Test of FSM and Counter	6
3.4.2	Full Test Bench	6
3.5	Coverage Metrics	7
3.6	Sign-off	7
3.7	Results	7
3.7.1	Unit Test	7
3.7.2	Full Test Bench - Short Run	9
3.7.3	Full Test Bench - Long Run	10
4	Targeting the Design to the Board	11
5	Demo	12
6	Total Number of Codeword Occurrences	13
7	Controlling the Clock	14
8	Utilization Report	14
9	Project Hierarchy	15

1 Implementation Notes

1.1 Adapting the LFSR

The following adaptations were made to the LFSR code from lab F:

- The bit width was extended from 23 to 24 bits.
- Thus, the tap had to be adjusted based on the documentation for LFSRs. A 24 bit maximum length LFSR uses the tap indices 24, 23, 22 and 17.
- The seed value was updated to fit the new specifications.

The seed value was calculated from the board number 49 and the last **two** digits of the student number (64).

	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1
<i>XNOR</i>	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	1	1	1	0

1.2 Finite State Machine

In order to create the FSM diagram, the "spine" for a successful '011000' codeword was drawn first and then the additional state transitions were added. Note that the output is associated with the state rather than the transition since this is a Moore state machine. The "spine" simply starts at a reset state and

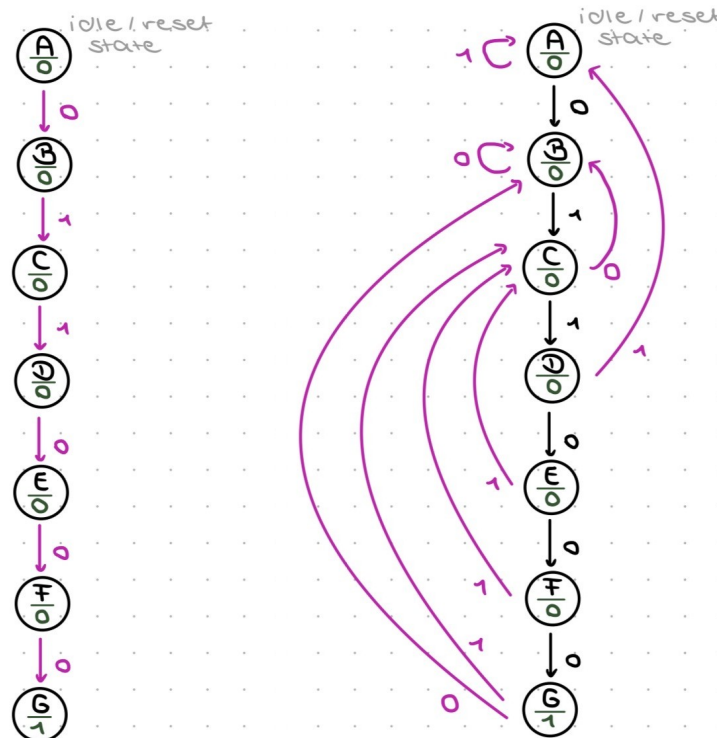


Figure 1: Finite State Machine Diagram to Detect Codeword '011000' with Overlap

follows the given codeword. The remaining "non-codeword" transitions lead to state B if a 0 is received, state C if a 1 is received after a 0 and state A if 3 ones

in a row are received. This is due to the condition that the sequence detector should accept overlapping codewords.

Next, the state machine design needed to be checked for minimization possibilities. This was done using a state table rather than an implication table since there are only 7 states.

current state	next state		state output
	input 0	input 1	
A	B	A	0
B	B	C	0
C	B	D	0
D	E	A	0
E	F	C	0
F	G	C	0
G	B	C	1

Table 1: State Table for Minimization

The only states with identical next states are B and G, but their outputs are different. Thus, no minimization is possible based on the state table.

The next step is to assign labels or names to the states. A Karnaugh map was used for this in order to ensure that states with many transitions between them have adjacent labels.

	00	01	11	10
0	A	B	-	-
1	D	-	-	-

Table 2: Karnaugh Map for Assigning State Names - Step 1

Since A is the idle or reset state, it was assigned '000'. States B and D are the only ones with direct transitions to or from A and were thus placed next to the idle state.

	00	01	11	10
0	A	B	G	-
1	D	C	-	-

Table 3: Karnaugh Map for Assigning State Names - Step 2

State C must be adjacent to B since there is 2 transitions between them. D can have either C, E or F as a neighbor while G must be next to C or B. Thus, C was placed between B and D and G was added next to B.

Finally, the remaining states were placed such that each is adjacent to at least one state it shares a transition with.

	00	01	11	10
0	A	B	G	F
1	D	C	E	-

Table 4: Karnaugh Map for Assigning State Names - Step 3

Thus, the states are assigned binary codes as follows:

$$A = (000)_2, B = (001)_2, C = (101)_2, D = (100)_2, E = (111)_2, F = (010)_2, G = (011)_2$$

(1)

'110' remains unassigned and thus not a valid state code.

1.3 Adapting the Counter

The original counter kept track of the number of zeros and ones. In this case, only a single variable is needed and thus the outputs were replaced by a single "num_codes" output. Originally, the "wrapping" of the sequence was detected by checking the sum of ones and zeros, but in this case the "max_tick" output of the LFSR was used to achieve this.

Additionally, the maximum size was adjusted based on the new bit width of 24.

```
module bit_out_counter(  
    input clk, sh_en, rst, bit_out, max_tick,  
    output [23:0] num_codes  
);  
  
reg [23:0] count_reg;
```

Figure 2: Adapted Counter Module Output

1.4 A Note on Clocks

From earlier labs there is a clock.v module available which uses the internal default clock at 100MHz to produce lower frequency signals. However, for simulation purposes it is easiest to use a fast clock, while a slower one should be selected for displaying the outputs using LEDs. This is due to the limited reaction time of the human eye.

Thus, the clock.v module was added to the sequence detector, but the scaling factor was changed between simulation and programming of the board. Thus, a scaling factor of 25000000 (2Hz) was chosen for the hardware application but a faster block was implemented during testing.

2 Writing the Top Module

The top module only includes instantiations of the previously discussed modules. Its structure is shown in the functional block diagram (see Figure 3).

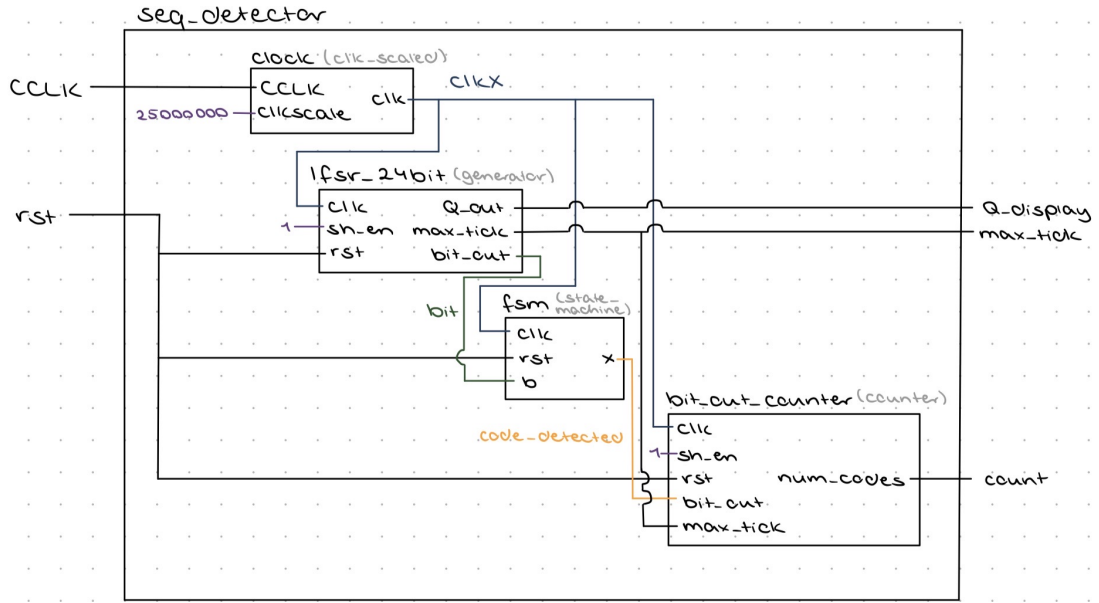


Figure 3: Sketch of the Functional Diagram

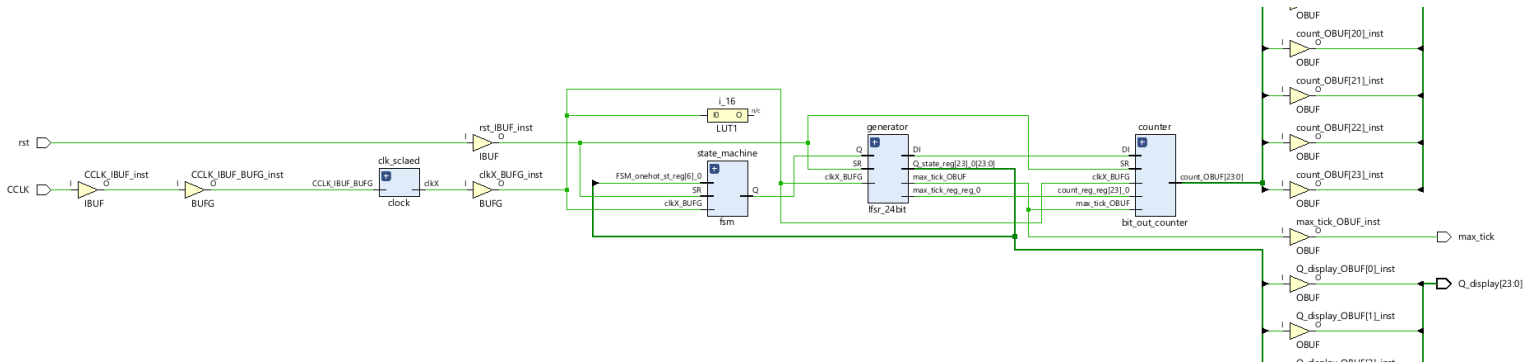


Figure 4: Schematic Generated in Vivado to Compare to Functional Diagram

The same blocks can be found in the Vivado schematic shown in Figure 4. Not all output bits of $Q_display$ and $count$ are included in the screenshot for readability purposes.

3 Test Plan

3.1 Scope

The sequence detector will be tested functionally to determine whether it responds to the given codeword accordingly. The clock and LFSR modules have already been tested extensively, with the former being provided and the latter having been analyzed closely in Lab F. Thus, the goal of this test plan is to verify the functionality of the FSM and counter together and then simulate the top module.

3.2 Methodology

In order to test the FSM and counter, a separate test bench is written. It contains a manually set, 36-bit sequence as inputs to the FSM to control the

number of codeword occurrences. Reset and "max_tick" signals are also manually varied. An additional clock cycle delay was added between test vectors to account for the delay from the Moore machine's state transition to its output change. To test the top module, a "full" test bench is firstly run for approx. 500 clock cycles. It only contains the instantiation of the sequence detector and a test clock. Secondly, the whole sequence is run and the total number of codeword occurrences is recorded through print statements.

3.3 Features of Test Bench

For the initial FSM and counter test, the number of codewords contained was varied (5 and 3) and in vector 1.1 the sequence ends with a codeword, while it does in vector 1.2. Additionally, both max_tick and rst were set to 1 for a clock cycle to test whether the counter would reset.

Since the sub modules have been tested extensively, rst is only set high at the start and end of the **full test bench** simulation and no other inputs are varied. It is possible to switch between short and long runs of the simulation by commenting out the corresponding code block.

3.4 Test Cases

3.4.1 Initial Test of FSM and Counter

- vector 1.1: A 36-bit sequence with 5 codewords, 2 of which overlap. The sequence ends with a codeword. rst and max_tick remain low.
sequence: 010101011000011000110001100010011000
expected outcome: A final count of 5.
- vector 1.2: A 36-bit sequence with 3 codewords, none of which overlap. The sequence does not end with a codeword. rst and max_tick remain low.
sequence: 000011000111011000000111011000101010
expected outcome: A final count of 3.
- vector 2: The same sequence from vector 1.1 was used but max_tick was set high for one clock cycle during the test.
expected outcome: The counter gets to 1 and then resets before finally reaching 4.
- vector 3: The same sequence from vector 1.1 was used but rst was set high for one clock cycle during the test.
expected outcome: The counter gets to 1 and then resets before finally reaching 4.

3.4.2 Full Test Bench

- Short Test: The sequence detector is run for 500 clock cycles after resetting for 2 cycles. The waveform is recorded.
- Long Test: The sequence detector is run slightly longer than $2^N - 1 = 2^{24} - 1$ to observe the total number of codeword occurrences and ensure max_tick behaves as intended. The sequence has a length of $2^{24} - 1$. However, the clock of the LFSR is a scaled version of the CCLK and thus has a longer period.

3.5 Coverage Metrics

Both high and low states for the rst and max_tick signals are tested. However, only 2 36-bit sequences are used. Thus, future tests should include a larger number of differing input sequences. The main edge case considered is an input which ends with the codeword.

Both a short excerpt and the full sequence are analyzed to ensure full functionality of the top module.

3.6 Sign-off

The unit tests are deemed successful if not only the final count is correct for each vector, but the counter also increases or resets at the appropriate times. Thus, both waveforms and console print statements are included in the results. The full test is deemed successful if the counter increases at the correct time in the wave form and the total number of occurrences is found to be reasonable.

3.7 Results

There are two separate files for the unit and full test benches, with the latter containing short and long versions which can be un-commented to run the two variations.

```
initial begin
    // uncomment the following line for the short run
    //$display("Short Simulation");
    // uncomment the following line for the long run
    $display("Long Simulation");
    rst = 1;
    repeat (2) @ (posedge CCLK);
    rst = 0;
    // uncomment the following line for the short run
    //repeat (500) @ (posedge CCLK);
    // uncomment the following line for the long run
    repeat (67108950) @ (posedge CCLK);
    rst = 1;
    repeat (5) @ (posedge CCLK);
    $finish;
end
```

Figure 5: Long and Short Versions of the Full Test Bench

3.7.1 Unit Test

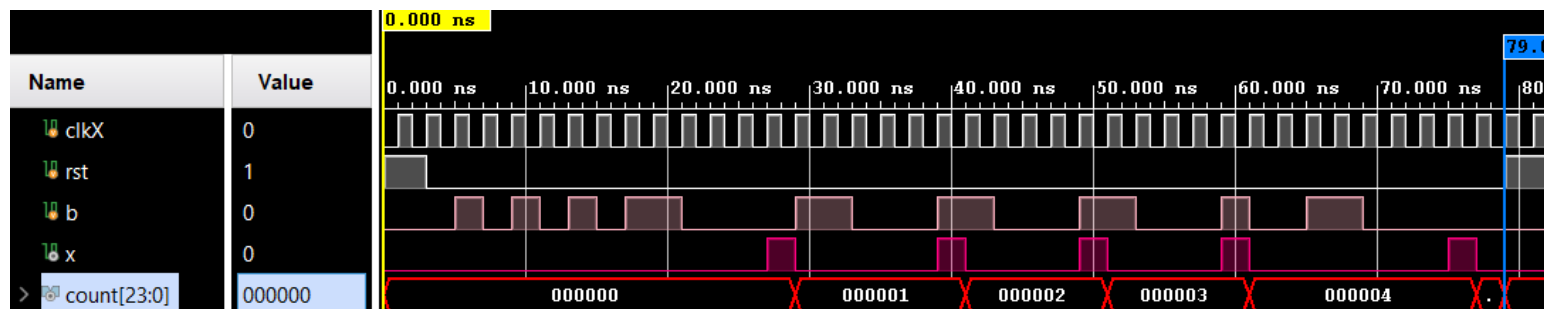


Figure 6: Waveform of Sequence 1.1 of the Initial Test

```

Sequence 1.1
Count is 0 at time          0
Count is 1 at time        29000
Count is 2 at time        41000
Count is 3 at time        51000
Count is 4 at time        61000
Count is 5 at time        77000

```

Figure 7: Console Output of Sequence 1.1 of the Initial Test

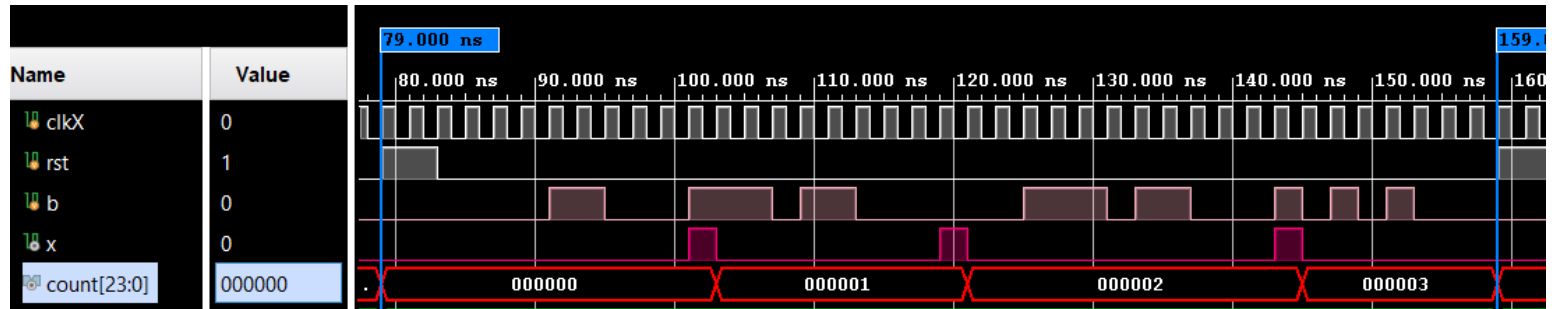


Figure 8: Waveform of Sequence 1.2 of the Initial Test

```

Sequence 1.2
Count is 0 at time        79000
Count is 1 at time       103000
Count is 2 at time       121000
Count is 3 at time       145000

```

Figure 9: Console Output of Sequence 1.2 of the Initial Test

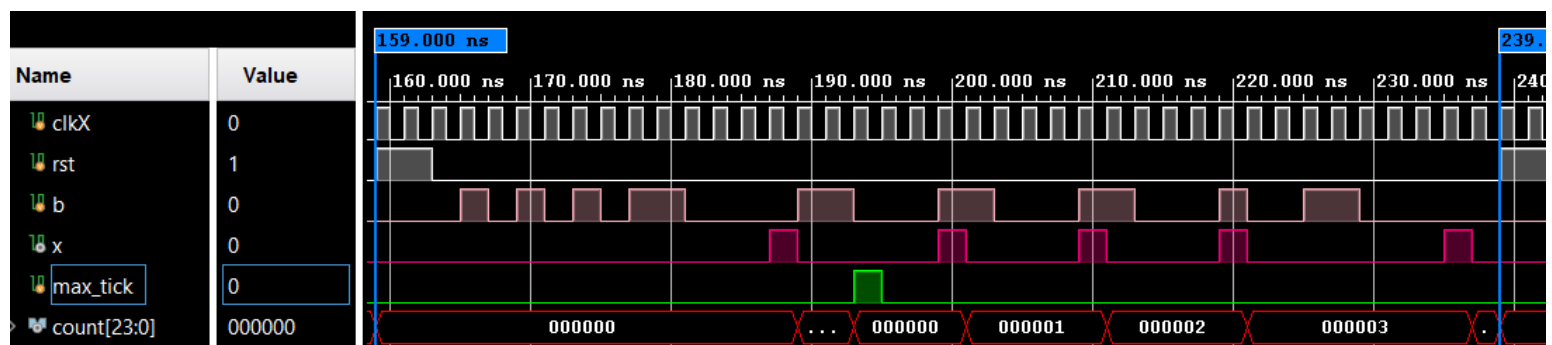


Figure 10: Waveform of Sequence 2 of the Initial Test

```

Sequence 2
Count is 0 at time                159000
Count is 1 at time                189000
codes 1
Count is 0 at time                193000
Count is 1 at time                201000
Count is 2 at time                211000
Count is 3 at time                221000
Count is 4 at time                237000

```

Figure 11: Console Output of Sequence 2 of the Initial Test

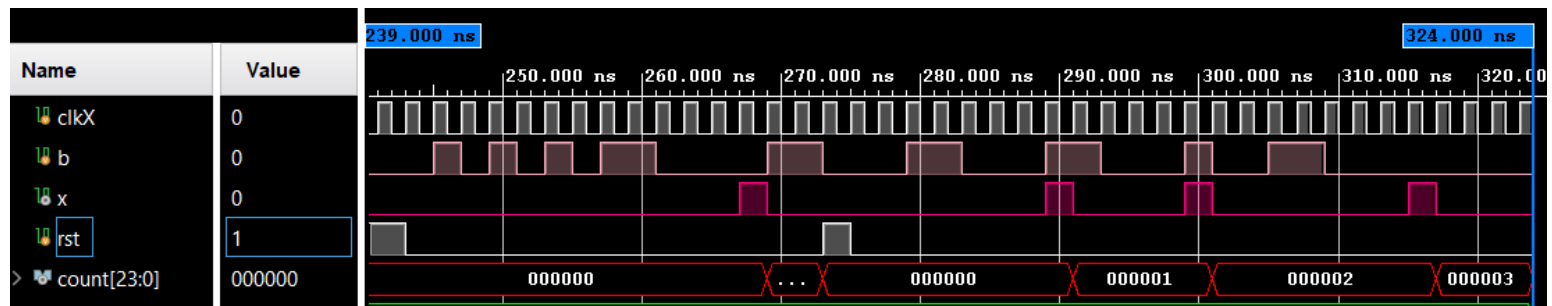


Figure 12: Waveform of Sequence 3 of the Initial Test

```

Sequence 3
Count is 0 at time                239000
Count is 1 at time                269000
Count is 0 at time                273000
Count is 1 at time                291000
Count is 2 at time                301000
Count is 3 at time                317000

```

Figure 13: Console Output of Sequence 3 of the Initial Test

3.7.2 Full Test Bench - Short Run

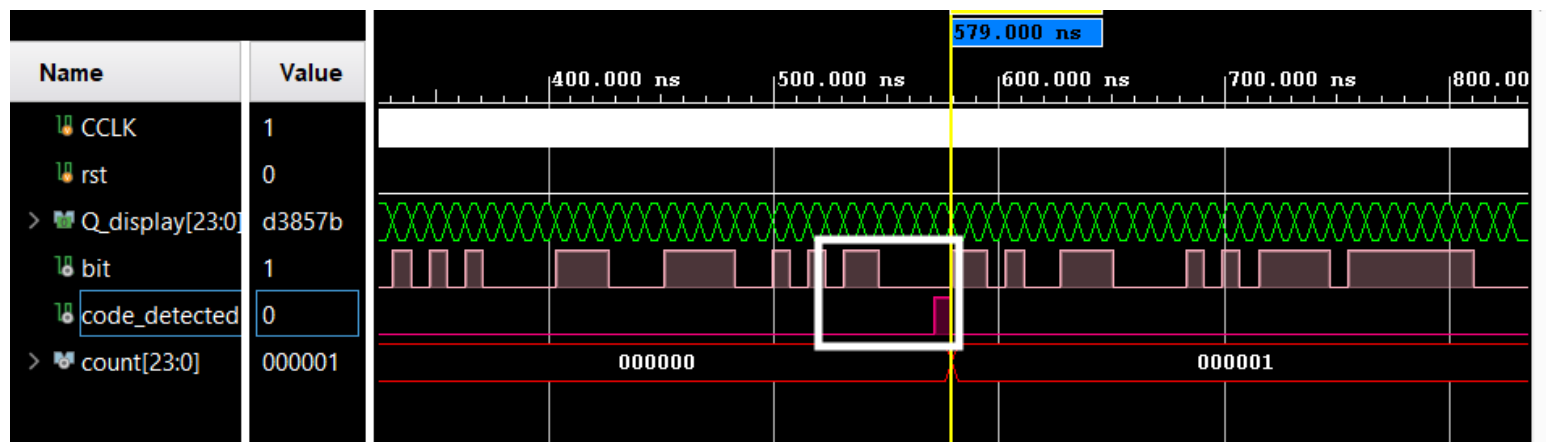


Figure 14: Waveform of the Short Top Module Test

The white rectangle in Figure 14 shows the codeword (011000). The code is detected as intended (see "code_detected") and the counter is increased. This waveform also confirms that it is necessary to allow for an additional clock cycle so that the signal can propagate.

```
Short Simulation
Count is 0 and max_tick is 0 at time 0
Count is 1 and max_tick is 0 at time 579000
```

Figure 15: Console Output of the Short Top Module Test

3.7.3 Full Test Bench - Long Run

Note: In order to reproduce the results in this section, the simulation time must be manually adjusted to allow the full simulation to run.



Figure 16: Manual Runtime adjustment for the Long Version of the Full Test Bench

The total number of occurrences throughout the entire sequence is **262144**.

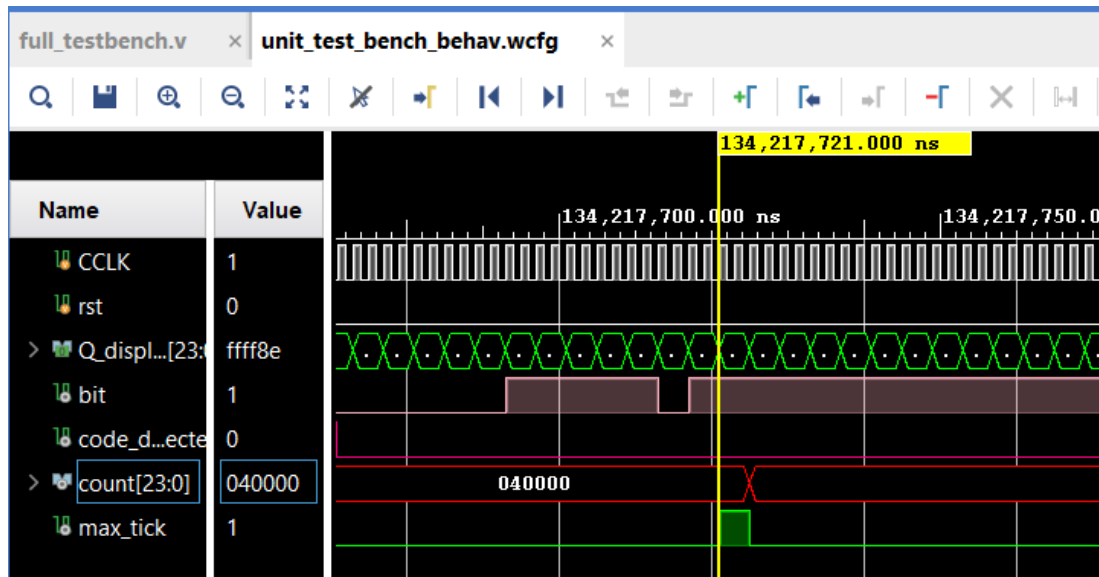


Figure 17: Waveform of the Long Top Module Test

```
Count is 262139 and max_tick is 0 at time 134216077000
Count is 262140 and max_tick is 0 at time 134216285000
Count is 262141 and max_tick is 0 at time 134216337000
Count is 262142 and max_tick is 0 at time 134216541000
Count is 262143 and max_tick is 0 at time 134217201000
Count is 262144 and max_tick is 0 at time 134217253000
Count is 262144 and max_tick is 1 at time 134217721000
codes 262144
Count is 0 and max_tick is 0 at time 134217725000
```

Figure 18: Console Output of the Long Top Module Test

The probability of any 6 bits sequence corresponding to the codeword is 0.5^6 and a sequence of length x would contain $x-5$ overlapping 6-bit sequences. Since the length x is $2^{24} - 1$, the expected number of occurrences can be estimated to be around $((2^{24} - 1) - 5) * 0.5^6 = 262143.9063 \approx 262144$. Thus, the simulation aligns exactly with the expected count and the module can be signed off on.

4 Targeting the Design to the Board

It was decided to display the pseudo random bit sequence and `max_tick` using the LEDs and show the counter on the 7 segment display. Thus, a multiplexer sub-module was added to the top module to convert the 32-bit counter value into 7 segment inputs.

Since the maximum decimal value of a 32-bit variable requires 10 decimal digits, but the display only has 4 and the counter does not go past 6 digits (262144). Thus, the last 4 digits of the counter are displayed by default and the next 4 digits can be shown by flipping a switch.

Note: The changes made to `q_out` and `count` mean that it is not a direct output of the full test bench anymore, but the counter can still be checked through the `tcl print` statements. To recreate the test bench result, the clock conversion factor must be changed to 1.

```
module seq_detector(
    input CCLK, rst, sel_digits,
    output max_tick,
    output [14:0] Q_display,
    output [3:0] enable,
    output [7:0] sseg
);
```

Figure 19: Adjusted Sequence Detector Declaration for Hardware Programming

Similarly to lab F, there are not enough LEDs to show the full bit sequence value, but since the bits shift anyways, it is sufficient to display the last 14 bits and use the 15th LED for `max_tick`.

5 Demo

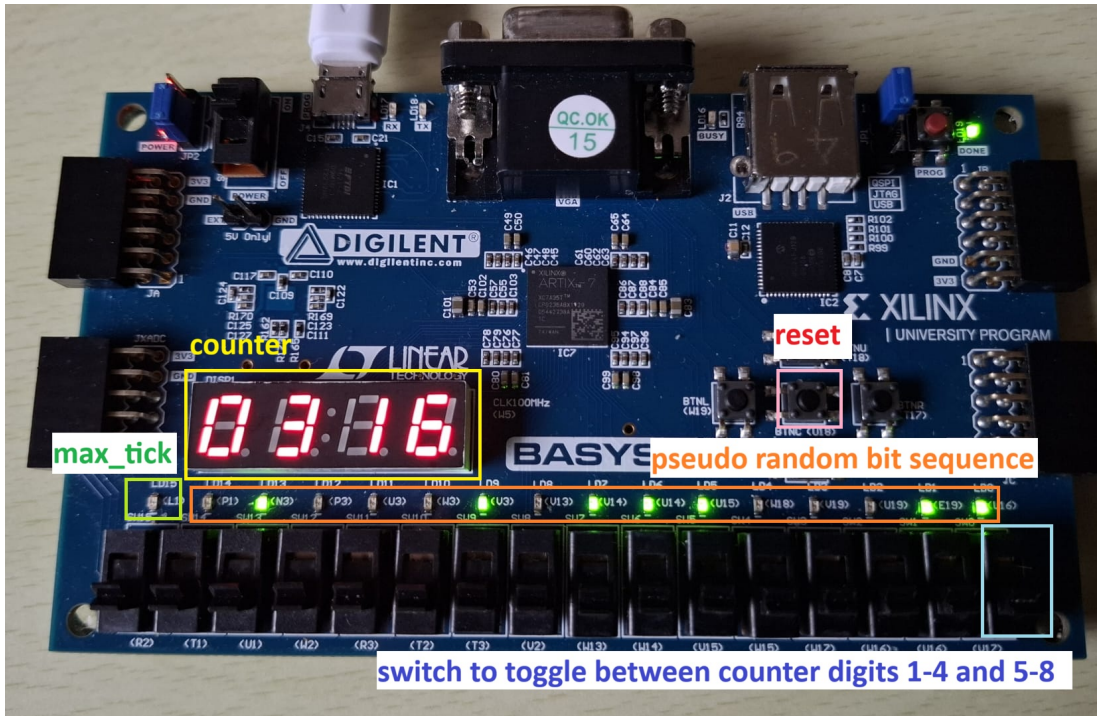


Figure 20: Allocation of Display, LEDs and Switch on the Basys 3 Board

Figure 20 shows the allocation of inputs and outputs on the board. The sequence runs at a frequency of 5 Hz such that it is visible to the human eye. The counter is displayed on the 7 segment display and the rightmost switch can be used to view digits 5 to 8 (assuming digit 1 is the least significant). The 15 LEDs are allocated to the pseudo random bit sequence (LEDs 0 to 14) and max_tick (LED 15).

The middle button is used to reset the sequence detector. The reset button and display selection switch are shown in Figures 21 and 22.

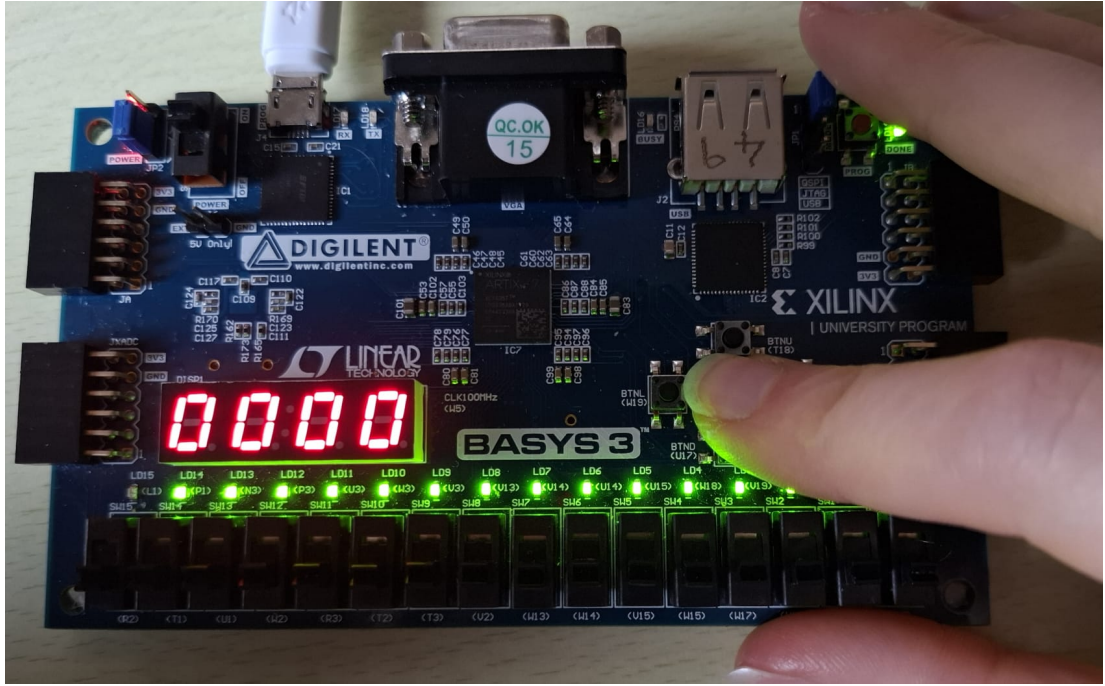


Figure 21: Response of the Board to Pressing the Reset Button

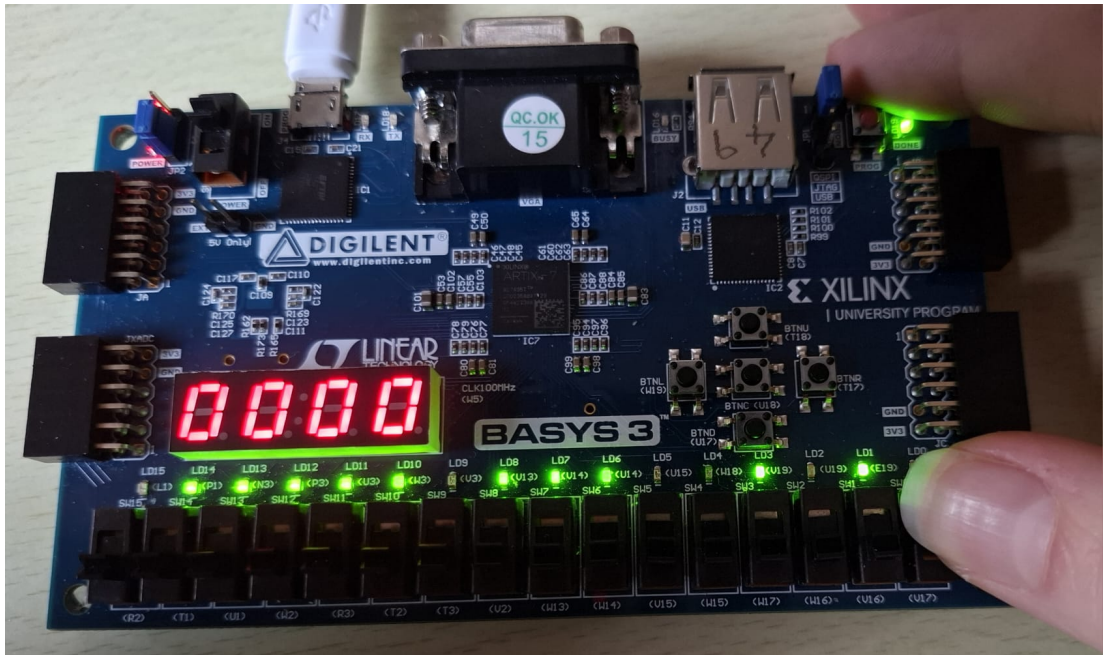


Figure 22: Response of the Board to Toggling the Display Selection Switch

6 Total Number of Codeword Occurrences

As stated previously, the total number of codeword occurrences is **262144** which aligns with the calculated expected value. The "Demo" section shows that the decimal digits 1-4 and 5-8 of the counter are displayed on the 7 segment display, based on whether switch 0 is high or low. Technically the counter has up to 10 decimal digits (32 bits), but with a maximum of 262144, 8 digits are more than sufficient.

7 Controlling the Clock

As explained in earlier sections, there is a clock module taking the internal clock on port W5 as an input and returning a 5 Hz clock signal (conversion factor of 10,000,000).

```
32 | set_property PACKAGE_PIN W5 [get_ports CCLK]
```

Figure 23: Declaration of Input Clock in XDC File

```
module seq_detector(
    input CCLK, rst, sel_digits,
    output max_tick,
    output [14:0] Q_display,
    output [3:0] enable,
    output [7:0] sseg
);
    wire bit;
    wire code_detected;
    wire clkX;
    wire [23:0] count;
    wire [23:0] Q_display_temp;

    clock clk_scaled(.CCLK(CCLK), .clk_scale(10000000), .clk(clkX));
```

Figure 24: Instantiation of Clock Module

8 Utilization Report

The "Register as Flip Flop" section of the utilization report is shown in Figure 25.

Tcl Console

Messages

Log

Reports

Design Runs

Utilization

Timing

Q

≡

⚙

Q

≡

⚙

%

Register as Flip Flop

Hierarchy

Summary

▼ Slice Logic

▼ Slice LUTs (24%)

LUT as Logic (24%)

▼ Slice Registers (<1%)

Register as Flip Flop (<1%)

Memory

DSP

Name

Used

▼ N seq_detector

107

ⓘ

clk_scaled (clock)

33

ⓘ

generator (lfsr_24bit)

25

ⓘ

counter (bit_out_counter)

24

ⓘ

mux_digits (mux)

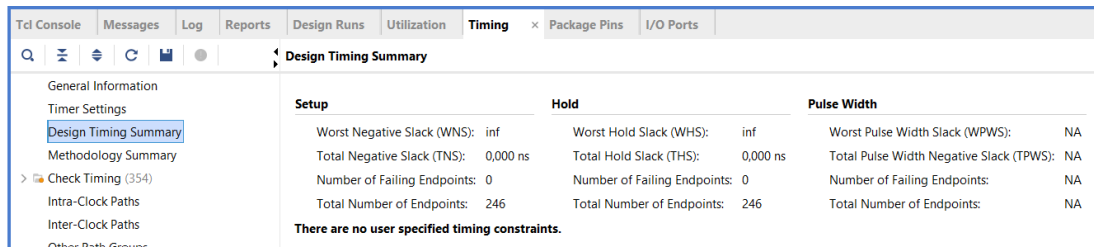
18

ⓘ

state_machine (fsm)

7

Figure 25: "Register as Flip Flop" Section of the Utilization Report



Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	inf	Worst Hold Slack (WHS):	inf	Worst Pulse Width Slack (WPWS):	NA
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	0,000 ns	Total Pulse Width Negative Slack (TPWS):	NA
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	NA
Total Number of Endpoints:	246	Total Number of Endpoints:	246	Total Number of Endpoints:	NA

There are no user specified timing constraints.

Figure 26: Timing Summary Report

9 Project Hierarchy

The file hierarchy of the design, simulation and constraint files is shown in Figure 27.

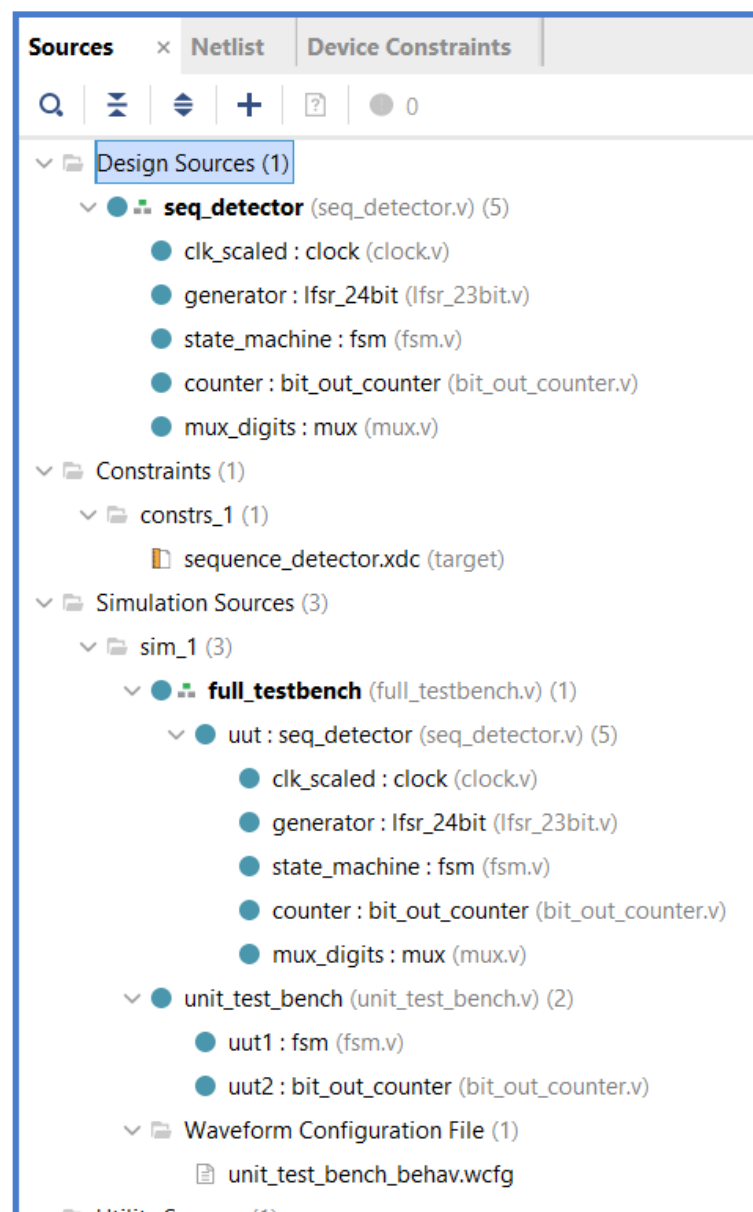


Figure 27: Project Hierarchy of Code Files